

Construction de données, pt.2

TD2 de G.I.M. sur Unicode

Ilaine Wang

25 novembre 2025

Ilaine Wang

Construction de données, pt.2

Unicode

Définitions

- ▶ Unicode, le **consortium** : organisation privée sans but lucratif qui coordonne le développement du standard Unicode.
- ▶ Unicode, un **standard** : standard informatique créé en 1987 qui permet des échanges de textes dans différentes langues et à un niveau mondial, maintenu par le Consortium Unicode.
- ▶ Unicode, un **catalogue** : véritable base de données sur les *caractères*, couvrant tous les systèmes d'écriture du monde (anciens et modernes), liés aux langues (*scripts*) ou non (symboles, emojis...).

(Très très) brève histoire

- ▶ fusion de deux projets (Unicode, et ISO/IEC 10646) en un seul en 1992 : actuellement conformité absolue entre ces deux standards
- ▶ septembre 2025 : publication de la version 17.0.0
 - ◊ 159,801 caractères (v13 = 143 859, v14 = 144 697, v15.0 = 149 186, v15.1 = 149 813, v16 = 154 998)
 - ◊ 172 scripts (v13 = 154, v14 = 159, v15.1 = 161, v16 = 168)
 - ◊ plus d'informations sur : <https://www.unicode.org/versions/latest>

Ilaine Wang

Construction de données, pt.2

Précédemment...

- ▶ systèmes binaire et hexadécimal
- ▶ bit (*binary digit*) vs. byte vs. octet
 - ◊ octet = unité de stockage du caractère
 - ◊ octet = 2 chiffres hexadécimaux = 8 positions binaires = 256 valeurs
- ▶ ASCII (table 7 bits) et ses limites
- ▶ les encodages sur **8 bits**
 - ◊ Windows (ou DOS) 1250, 1252
 - ◊ MacRoman
 - ◊ ISO-8859-*n* et ses caractéristiques
- ▶ **codage**
 - ◊ **encodage** : saisie, sauvegarde, écriture...
 - ◊ **décodage** : interprétation, affichage, lecture...
- ▶ commandes
 - ◊ identification : `file`, `chardet`, `uchardet`
 - ◊ conversion : `iconv`

Ilaine Wang

Construction de données, pt.2

Unicode

Objectifs

- ▶ remplacer l'utilisation de pages de code nationales
- ▶ définir un jeu de caractères codés qui permettrait d'écrire dans tous les systèmes d'écriture du monde
- ▶ fournir un nom, mais aussi un code numérique unique à chaque caractère, le *code Unicode* : U+(X)XXXX

Principes

- ▶ universalité
- ▶ abandon de l'équation 1 caractère = 1 octet
 - ◊ non pas *une table*, mais une division en *plans*
 - ◊ potentialité de 1 114 112 caractères (17 plans de 2^{16} places, donc $17 \times 65\,536$)
- ▶ véritable élaboration théorique du concept de *caractère*

Ilaine Wang

Construction de données, pt.2

Partitionnement en 17 plans

▶ plan 0	0000 – FFFF	Plan Multilingue de Base (PMB/BMP)
▶ plan 1	10000 – 1FFFF	Plan Multilingue Complémentaire (PMC/SMP)
▶ plan 2	20000 – 2FFFF	Plan Sinographique Complémentaire (PSC/SIP)
▶ plan 3	30000 – 3FFFF	Plan Sinographique Ternaire (PST/TIP)
▶ plan 4-13	40000 – DFFFF	Plans réservés supplémentaires
▶ plan 14	E0000 – EFFFF	Plan Complémentaire Spécial (PCS/SSP)
▶ plan 15	F0000 – FFFFF	Plan supplémentaire à usage privé - A (privé A)
▶ plan 16	100000 – 10FFFF	Plan supplémentaire à usage privé - B (privé B)

- ▶ “réservés” = blocs à assigner, éventuellement, dans une version ultérieure d'Unicode
- ▶ “à usage privé” = blocs qui ne seront pas utilisés par le Consortium

Plan Multilingue de Base (ou *Basic Multilingual Plane*)

65 536 places qui se répartissent en 8 zones :

1. *General Scripts Area* de 0x0000 à 0x1FFF (8 172 places)
 - ◊ De tout sauf les caractères chinois (ou *sinogrammes*) et dérivés
2. *Symbols Area* de 0x2000 à 0x2FFF (4 096 places)
 - ◊ Symboles variés (ponctuation, flèches, mathématiques)
3. *CJK Ideographs Area* de 0x3000 à 0x3FFF (4 096 places)
 - ◊ Signes de ponctuation, syllabaires japonais, clefs pour les dictionnaires mais pas les sinogrammes eux-mêmes !
4. *CJK Unified Ideographs Area* de 0x4000 à 0x9FFF (24 576 places)
 - ◊ Masse imposante des sinogrammes...

Ilaine Wang

Construction de données, pt.2

Plan Multilingue de Base (ou *Basic Multilingual Plane*)

5. *Asian Scripts Area* de 0xA000 à 0xD7FF (14 336 places)
 - ◊ L'écriture Yi (minorité chinoise)
 - ◊ Le syllabaire basé sur l'alphabet coréen *hangeul*
6. *Surrogates Area* de 0xD800 à 0xDFFF (2 048 places)
 - ◊ Usage technique réservé au codage UTF-16
 - ◊ Ne représente aucun caractère
7. *Private Use Area* de 0xE000 à 0xF8FF (6 400 places)
 - ◊ Utilisable *ad libitum* par les applications informatiques
 - ◊ Ne fera jamais l'objet d'une normalisation
8. *Compatibility & Specials Area* de 0xF900 à 0xFFFF (1 792 places)
 - ◊ Fourre-tout dû au besoin de rester compatible avec les systèmes préexistants puisque tout ce qui a été codé doit pouvoir se coder en Unicode...

→ voir [PMB sur Wikipédia](#), article très complet

Ilaine Wang

Construction de données, pt.2

Ilaine Wang

Construction de données, pt.2

Plan Multilingue de Base (ou *Basic Multilingual Plane*)

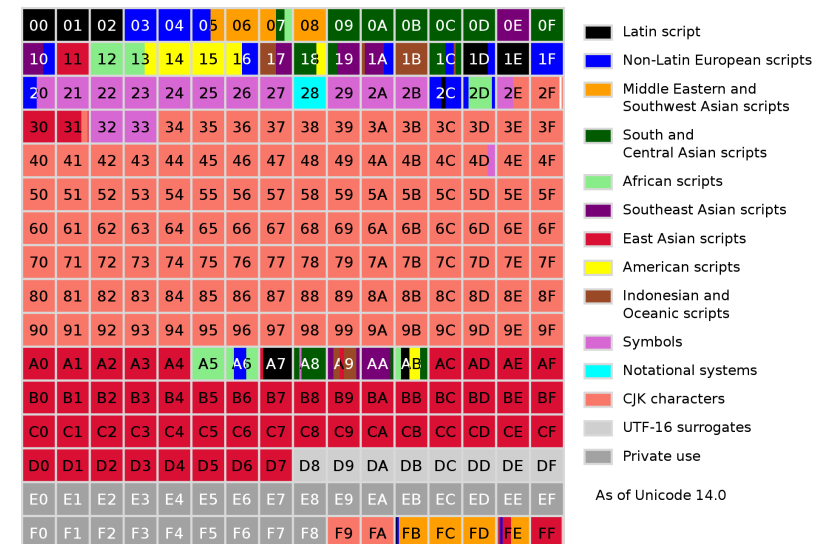


Figure 1: Plan Multilingue de Base

Ilaine Wang

Construction de données, pt.2

Les formats d'encodage : UTF (*Universal* ou *Unicode Transformation Format*)

Procédures permettant de représenter chaque code Unicode par une séquence d'octets... mais combien ?

- ▶ format à taille fixe (nombre d'octets fixe pour chaque caractère)
 - ◇ UTF-32
- ▶ formats à taille variable (nombre d'octets dépendant du caractère considéré)
 - ◇ UTF-8
 - ◇ UTF-16

Ilaine Wang

Construction de données, pt.2

Les formats d'encodage : UTF-8

Schémas des séquences d'octets UTF-8

- ▶ Si on a 1 octet :
 - ◇ celui commence par 0 (premier bit nul en ASCII !)
- ▶ Si on a plusieurs octets :
 - ◇ aucun octet ne commence par 0, pour éviter de confondre avec les octets ASCII ;
 - ◇ le premier commence par 110 s'il s'agit de 2 octets, 1110 s'il s'agit de 3 octets et 11110 s'il s'agit de 4 octets ;
 - ◇ les suivants commenceront par 10.

Numéro/code Unicode	Codage UTF-8
de U+0000 à U+007F	0xxxxxxx
de U+0080 à U+07FF	110xxxxx 10xxxxxx
de U+0800 à U+FFFF	1110xxxx 10xxxxxx 10xxxxxx
de U+10000 à U+1FFFFF	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

Ilaine Wang

Construction de données, pt.2

Les formats d'encodage : UTF-8

Principes

- ▶ l'unité de stockage est l'octet...
- ▶ ...mais le nombre d'octets utilisés pour représenter un caractère dépend de la position de ce caractère dans le catalogue Unicode.
 - ◇ les 128 premiers caractères sont représentés sur un seul octet (= compatible avec ASCII)
 - ◇ tous les autres caractères seront représentés par au moins 2 octets

Mise en oeuvre

```
01001001 01101100 00100000 01100011 01101111 01101110
01101110 01100001 11000011 10101110 01110100 00100000
01101100 01100001 00100000 01100011 11000011 10101001
01101100 11000011 10101001 01100010 01110010 01101001
01110100 11000011 10101001
```

- ▶ comment savoir combien d'octets on doit lire par caractère ?
- ▶ comment distinguer ces octets entre eux ?

Ilaine Wang

Construction de données, pt.2

Les formats d'encodage : UTF-8

Schémas des séquences d'octets UTF-8

Avec ce système, on réduit le nombre de *bits* ou *positions binaires utiles*... mais cela reste suffisant pour l'ensemble des caractères à encoder !

- ▶ Dans le cas de 2 octets :
 - ◇ 110xxxxx 10yyyyyy
 - ◇ 11 positions binaires utiles (5 + 6)
 - ◇ soit $2^{11} = 2\,048$ possibilités
- ▶ Dans le cas de 3 octets :
 - ◇ 1110xxxx 10yyyyyy 10zzzzzz
 - ◇ 16 positions binaires utiles (4 + 6 + 6)
 - ◇ soit $2^{16} = 65\,536$ possibilités
- ▶ Dans le cas de 4 octets :
 - ◇ 11110xxx 10yyyyyy 10zzzzzz 10tttttt
 - ◇ 21 positions binaires utiles (3 + 6 + 6 + 6)
 - ◇ soit $2^{21} = 2\,097\,152$ possibilités...

Ilaine Wang

Construction de données, pt.2

Les formats d'encodage : UTF-32

Définition

- ▶ l'unité de stockage est le *codet* de 32 bits, ou *quadruple octet*
- ▶ chaque caractère est donc représenté sur 4 octets dont :
 - ◊ le premier est nul (théoriquement maximum 0xFFFFFFFF mais nombre trop grand puisque Unicode est limité à 10FFFF (voir [la page Wikipédia des plans](#)))
 - ◊ le deuxième est également nul pour tous les caractères du PMB, le Plan Multilingue de Base (maximum 0xFFFF)

Qualités ?

- ▶ avantage : simplicité !
- ▶ inconvénient : lourd...

Le boutisme



Figure 2: Gulliver chez les Liliputiens ou, par quel bout manger les oeufs à la coque ?

Les formats d'encodage : UTF-16

Définition

- ▶ l'unité de stockage est le *doublet* de 16 bits, ou *double octet*
- ▶ chaque caractère est représenté par 1 ou 2 modules de 2 octets :
 - ◊ 1 module de 2 octets pour tous les caractères du PMB
 - ◊ 2 modules de 2 octets (donc 4 octets) pour les autres plans

Qualités ?

- ▶ avantage : moins lourd que UTF-32, surtout quand on ne sort pas ou peu du PMB
- ▶ inconvénient : un peu plus compliqué...

Définition

Le boutisme (*endianness* en anglais) désigne l'**ordre dans lequel ces octets sont placés et donc lus**. Il existe deux conventions opposées :

- ▶ l'orientation **gros-boutiste** (*big-endian*, BE) qui démarre avec les octets de poids forts,
- ▶ et l'orientation inverse **petit-boutiste** (*little-endian*, LE).

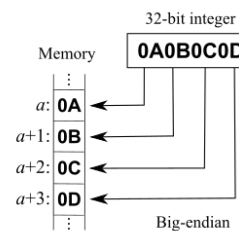


Figure 3: Big-endian

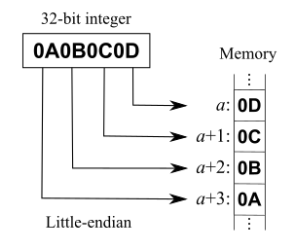


Figure 4: Little-endian

Le Byte Order Mark

L'indicateur d'ordre des octets pour les codages UTF.

Codage	Représentation hexa	Windows / ISO Latin 1
UTF-8	EF BB BF	ï»¿
UTF-16 BE	FE FF	þÿ
UTF-16 LE	FF FE	ÿþ
UTF-32 BE	00 00 FE FF	~@~@þÿ
UTF-32 LE	FF FE 00 00	ÿþ~@~@

~@ = caractère NUL (U+0000)

Attention le BOM n'est **pas indispensable** ! Tous les fichiers encodés dans un des codages Unicode n'ont pas forcément de BOM. Par ailleurs il n'est **pas recommandé** pour UTF-8.

Le Byte Order Mark

```
FE FF 00 4C 00 65 00 20 00 6C 00 61 00 63 00 20
00 43 00 68 01 6B 00 7A 00 65 00 6E 00 6A 00 69
00 20 00 28 4E 2D 79 85 5B FA 6E 56 00 29 00 20
00 65 00 73 00 74 00 20 00 75 00 6E 00 20 00 6C
00 61 00 63 00 20 00 6E 00 61 00 74 00 75 00 72
00 65 00 6C 00 20 00 64 00 27 00 65 00 61 00 75
00 20 00 64 00 6F 00 75 00 63 00 65 00 20 00 64
00 75 00 20 00 4A 00 61 00 70 00 6F 00 6E 00 2E
00 0A
```

Figure 6: BOM dans un fichier (2)

Le Byte Order Mark

```
FF FE 4C 00 65 00 20 00 6C 00 61 00 63 00 20 00
43 00 68 00 6B 01 7A 00 65 00 6E 00 6A 00 69 00
20 00 28 00 4E 2D 79 85 5B FA 6E 56 00 29 00 20 00
65 00 73 00 74 00 20 00 75 00 6E 00 20 00 6C 00
61 00 63 00 20 00 6E 00 61 00 74 00 75 00 72 00
65 00 6C 00 20 00 64 00 27 00 65 00 61 00 75 00
20 00 64 00 6F 00 75 00 63 00 65 00 20 00 64 00
75 00 20 00 4A 00 61 00 70 00 6F 00 6E 00 2E 00
0A 00
```

Figure 5: BOM dans un fichier (1)

Unicode : quelques références web

Pour identifier un caractère :

- le site officiel du Consortium Unicode :
<https://www.unicode.org/charts/>
 - ◇ identification par le code hexadécimal (sans 0x ou U+), par script, par nom (index alphabétique)
- le site Unicode-Table : <https://unicode-table.com/en/blocks/>
 - ◇ identification par nom, par bloc, par code hexadécimal
- le site Compart : <https://www.compart.com/fr/unicode/>
 - ◇ identification par bloc, par caractéristique, par classe